

The Way Kubernetes Works

Developers and administrators in the current IT-transformed world are well aware that container orchestration plays a vital role in day-to-day deployment or management scenarios. There are plenty of orchestration tools, including Kubernetes. In this article we take a quick look at the architecture of Kubernetes, and how a service account can be created in it.

A container management tool is capable of managing containers running on a single host, but not intelligent enough to manage containers running on multiple hosts, including HA, Replica, load-balancing of a pod that has a hosted application running, etc. To overcome these limitations, we can use Kubernetes as an orchestrator tool running on top of container management systems like Docker, PodMan or rkt.

Architecture of Kubernetes

As seen in Figure 1, Kubernetes has an architecture of three plus one clusters, in which three hosts are nodes (slaves) and one has the master server (manager), which takes care of the deployment and management of an application. The application, in the form of a pod, will get deployed on the nodes. Each node has Kubelet, which talks to the REST API of the master and container runtimes on the node. On each node, including the master, there is a network plugin running for the overlay network. In addition to these, *etcd*, *secheduler* and *controller* manifest on the master.

Isolation of the pod and service layer in Kubernetes

By default, Kubernetes uses Intranet networking to communicate between the node(s) and the pod, the pod and node(s), service to the pod(s) or pod(s) to the service. In Kubernetes, service is an object that takes endpoints as the pod(s) IP. It has a cluster IP, NodePort and LoadBalancer as type. By default, the cluster IP is not

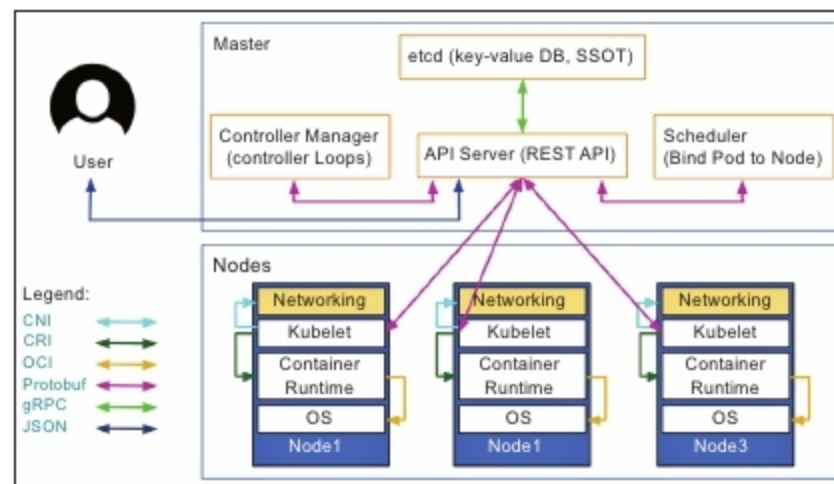


Figure 1: The 'three+one' cluster architecture

reachable from the outside world.

Service is an abstraction layer for security in Kubernetes, which isolates any public request directly hitting the pod. We can use Ingress, NodePort or LoadBalancer to access the application from the public world.

The security context in Kubernetes

By default, any pod run on Kubernetes can be secured by adding a security context to it. A security context defines privileges and access control settings for a pod or container.

The following is an example of a pod using a security context.

```
pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000
```

```
runAsGroup: 3000
fsGroup: 2000
volumes:
- name: myvol
  emptyDir: {}
containers:
- name: sec-ctx-demo
  image: busybox
  command: [ "sh", "-c",
    "sleep 1h" ]
  volumeMounts:
  - name: myvol
    mountPath: /data/demo
securityContext:
  allowPrivilegeEscalation: false
```

In the above example, the container runs any process as a UID (UserID) 1000, and a GID (Group ID) 3000. The file system */test/data* will have a GID 2000, the default being 0. This way, one can override the default configuration of a pod and achieve the next level of security.

ServiceAccount in Kubernetes

Kubernetes uses RBAC (role based access control) for getting access to clusters, using either ClusterRole or Role. Let us look at how a service account can be created using YAML.

```
cat <<EOF | kubectl create -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: user1
  namespace: kube-system
EOF
```

END 

By: Ashutosh S. Bhakare

The author is an open source enthusiast, a Docker community leader, an active Fedora contributor, as well as a CKA, CKAD and Red Hat/SUSE certified instructor with 19 years of experience as a trainer and consultant.